WEEK 1 & 2

DEVOPS

1. What is DevOps?

DevOps is a **modern way of developing software** where the:

- **Development team** (writes code)
- **Operations team** (deploys & manages servers)
- **Testing team** (checks quality)

work **together as a single team** instead of separately.

Why?

Because modern applications need:

- Faster updates
- Fewer bugs
- Quick bug fixing
- Stable performance
- Zero downtime

Traditional method (Waterfall) was slow and error-prone.
DevOps solves these problems.

EX: Think of a College Cultural Event

Without DevOps (Traditional):

- Cultural team plans program
- Light & sound team comes later
- Stage setup team comes later
- Event takes days to prepare
- Miscommunication → mistakes

With DevOps:

- All teams sit together and prepare stage at the same time
- Much faster
- Fewer mistakes
- Better coordination

This is exactly how DevOps works in software.

2. Why DevOps Emerged?

Modern companies like Google, Amazon, Netflix:

- release updates **daily**
- serve **millions of users**
- cannot afford downtime
- have complex architectures

Old methods failed because:

- Code was tested only at the end
- Deployment took hours/days
- Developers and operations never talked
- Fixing issues took a long time
- Bugs appeared in production

DevOps solves all of this.

REAL-WORLD EXAMPLES

Example 1 — Instagram Feature Update

User interface changes or new filters appear frequently.

How?
→ Developers push code
→ Jenkins builds automatically
→ Tests run
→ Docker deploys new version
→ Kubernetes manages containers

Fast, automated pipeline = DevOps

Example 2 — Online Exam Portal Slowdown

During exams → traffic increases.
Without DevOps → server crashes.
With DevOps → monitoring detects high CPU → auto-scales servers.

Less downtime, more reliability

3. Core Principles

1. Collaboration

All teams share tools, processes, knowledge.

Why important?

When developers and operations work separately:

- Communication gaps
- Delays
- Errors

When they collaborate:

- Problems are solved faster
- No blaming
- Common goals

Example:

Developer commits code → tester reviews → ops deploy → all discuss feedback immediately.

2. Automation

Automation removes manual mistakes.

Automation applies to:

- Code build
- Testing
- Deployment
- Monitoring
- Environment setup

Example 1:

Without automation:
Tester manually tests login page daily.

With automation:
Selenium tests login page automatically.

Example 2:

Without automation:
Ops manually deploys .war file every release.

With automation:
Jenkins deploys .war file automatically.

3. Continuous Integration (CI)

Developers integrate (merge) their code frequently (often many times a day).

Why important?

Because if 10 developers work for 10 days without merging:

- Code conflicts occur
- Bugs remain hidden
- Integration becomes painful

CI catches errors early.

Example:

If one developer's code breaks the build → Jenkins alerts instantly → quick fix.

4. Continuous Delivery (CD)

CD ensures that every successful build can be deployed at any time.

Why important?

Because manual packaging is slow.

Example:

After CI, Jenkins automatically:

- Builds project
- Packages .jar/.war
- Sends it to testing environment

App is always ready for deployment.

5. Continuous Deployment

Every change that passes all tests is deployed to production automatically.

Why important?

Companies want features delivered quickly.

Example:

Amazon deploys new features thousands of times per day.

6. Continuous Monitoring

Monitoring tools track:

- server health
- performance
- errors
- crashes

Why important?

Because issues must be detected before users complain.

Example:

If login API is slow → monitoring tool alerts DevOps team → fix deployed.

4. DevOps Culture

DevOps is not only tools — it is **behaviour, mindset, culture**.

1. Shared Responsibility

Everyone owns the product outcome.

Example:

If app crashes:

- Developer → checks code
- Ops → checks servers
- Tester → checks functionality
  Together → fix quickly

2. Fail Fast, Learn Fast

Failures are used for improvement, not blame.

Example:

A deployment failed because of wrong config.
Instead of blaming → team creates automated config checker.

3. Transparency

Everyone knows what others are doing.

Example:

All changes visible in GitHub
All builds visible in Jenkins
All deployments visible in logs

This avoids confusion.

5. Key Elements of DevOps

1. Continuous Integration (CI)

Example 1:

Java project → developer pushes code → Jenkins compiles → detects missing semicolon.

Example 2:

Python project → unit tests fail → CI pipeline stops → avoids production issues.

2. Continuous Delivery (CD)

Example 1:

Maven automatically packages .jar file after CI.

Example 2:

CD pipeline deploys latest version to testing server daily.

3. Continuous Deployment

Example 1:

Bug fix in login module goes live immediately.

Example 2:

New banner added in website → gets auto-deployed.

4. Infrastructure as Code (IaC)

Example 1:

Terraform creates 10 servers using code.

Example 2:

Ansible configures 50 servers identically.

5. Continuous Monitoring

Example 1:

Grafana shows CPU spikes → server auto-scales.

Example 2:

Prometheus detects memory leak → alerts team instantly.

6. DevOps Tools

Git (Code Versioning)

Example: storing attendance management system code.

Jenkins (Automation / CI-CD)

Example: auto-building student portal.

Docker (Containers)

Example: packaging Java app with dependencies.

Ansible (Automation)

Example: installing Java on 50 servers.

Grafana (Monitoring)

Example: tracking CPU, RAM during exam peak time.